

¿Qué son los Hooks de Rendimiento? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

Los hooks de rendimiento son herramientas nativas de React que optimizan el ciclo de renderizado evitando re-renderizar componentes. Los dos principales son:

useMemo: utilizado para “memorizar” el resultado de una función costosa de calcular

useCallback: memoriza la instancia de una función. En JavaScript, cada render crea nuevas referencias de funciones en memoria, entonces, useCallback mantiene la misma referencia en memoria a menos que cambien sus dependencias. Sirve para pasar callbacks a componentes hijos optimizados con React.memo, evitando que sospechen que sus props cambiaron.

```
import React, { useState, useMemo, useCallback } from 'react';

// Componente secundario optimizado: no se re-renderiza si sus props no cambian
const BotonAccion = React.memo(({ onClick, children }) => {
  console.log('Renderizando botón: ${children}');
  return <button onClick={onClick}>{children}</button>;
});

export default function ContadorRendimiento() {
  const [contador, setContador] = useState(0);
  const [texto, setTexto] = useState('');

  // Evitamos recalcular la operación pesada al escribir en el input
  const calculoPesado = useMemo(() => {
    console.log('Ejecutando cálculo costoso...');
    let total = 0;
    for (let i = 0; i < 50_000_000; i++) {
      total += 1;
    }
    return contador * 2;
  }, [contador]);

  // Se mantiene la referencia estable de la función
  const incrementar = useCallback(() => {
    setContador((c) => c + 1);
  }, []); // Referencia constante a lo largo de toda la vida del componente

  return (
    <div>
      <h3>Resultado pesado: {calculoPesado}</h3>
      <p>Contador: {contador}</p>

      { /* BotonAccion no se re-renderiza cuando cambia 'texto' gracias a useCallback */ }
      <BotonAccion onClick={incrementar}>Incrementar</BotonAccion>

      <div style={{ marginTop: '1rem' }}>
        <input
          type="text"
          value={texto}
          onChange={(e) => setTexto(e.target.value)}
          placeholder="Escribe aquí para probar renders..."
        />
      </div>
    </div>
  );
}
```

En este ejemplo podemos ver que en caso de no utilizar `useMemo` cada vez que el usuario escribe en el input el estado del texto cambia y se reevalúa el componente completo, haciendo que el bucle de 50 millones de iteraciones se ejecute con cada entrada que ponga el usuario, en cambio, utilizando `useMemo` simplemente se omitiría el bucle porque contador no cambió. Además utilizamos `useCallback` para poder evitar que por cada re-render del input se cree una función nueva en memoria porque se conserva su identidad referencial.

¿Qué es el contexto en React? ¿Cuáles son sus características y para qué sirve? ¿Qué diferencia existen con los props? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

Es una manera de pasar datos mediante la jerarquía de componentes sin la necesidad de pasar props manualmente en cada nivel. Sirve para solucionar el problema del prop drilling (pasar props desde el componente padre a un hijo muy profundo en la jerarquía sin que los otros niveles lo necesiten). Para crear un contexto utilizamos el hook `createContext` y para poder utilizarlo debemos envolver el árbol de componentes con el componente `Provider`. Por último, para consumirlo debemos usar el hook `useContext`.

```
import { createContext, useContext, useState } from 'react';

// 1. Crear el contexto
const ThemeContext = createContext();

// 2. Proveer el valor
function App() {
  const [theme, setTheme] = useState('dark');

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Header />
    </ThemeContext.Provider>
  );
}

// 3. Consumir el valor sin importar los niveles intermedios
function Header() {
  return <ThemeButton />;
}

function ThemeButton() {
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <button onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
      Tema actual: {theme}
    </button>
  );
}
```

En este ejemplo podemos ver como creamos un contexto llamado ThemeContext, el cual sirve para cambiar el tema de la página web de claro a oscuro. Dentro de la función App se define su estado poniendo theme en dark y su función actualizadora setTheme. <ThemeContext.Provider> es el componente que emite los datos. En su prop se pasa un objeto con { theme, setTheme } haciendo que todo componente que esté envuelto dentro de este proveedor, ya sea directa o indirectamente, tiene acceso a dicho valor. useContext(ThemeContext) se conecta directamente al contexto más cercano de ThemeContext y extrae { theme, setTheme } y muestra el tema actual de la página. Al hacer clic en el botón, se ejecuta setTheme(...) alternando el valor entre light y dark. Al cambiar el estado en App, el proveedor notifica el cambio y ThemeButton se vuelve a renderizar automáticamente con el nuevo valor.